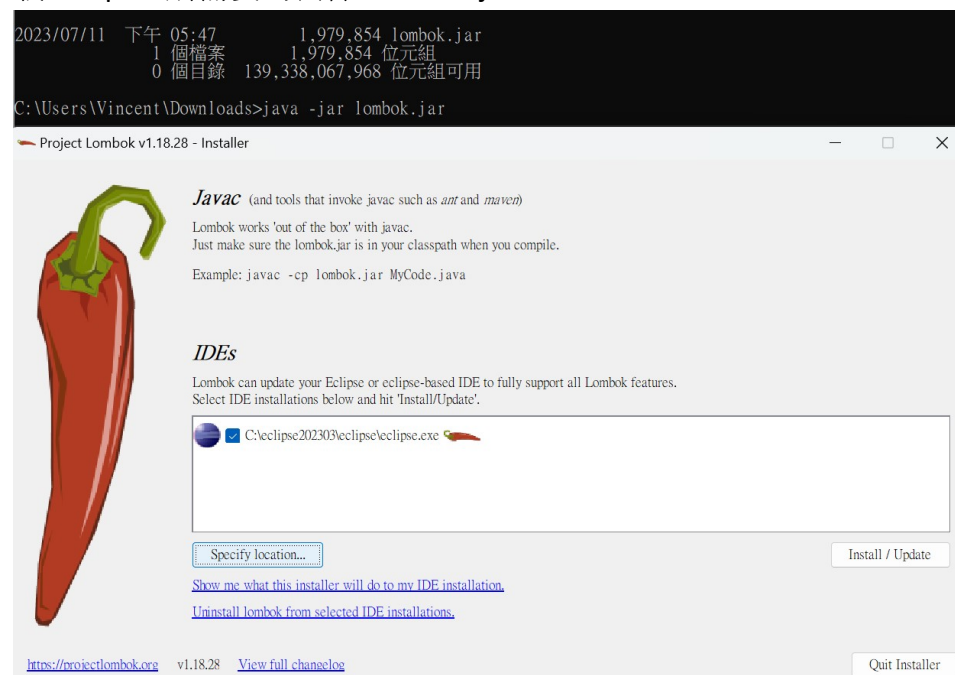


Eclipse 安裝 lombok – 讓撰寫 JAVA 程式更加優雅簡潔

Lombok 提供了簡單的標注來幫助我們簡化消除一些必須有但是臃腫的 Java 程式碼，比如：`getter`、`setter` 和 `Constructor` (建構函數)…等，這些函數一般我們通過 IDE，如：`Eclipse` 自動生成，有了 `lombok` 只要用 `Annotation` (標注) 創建一些程式碼，就能輕鬆搞定，並以減少重復程式碼的數量，這可以讓 Java 的程式更加優雅簡潔。

使用 Lombok 的完整範例，展示了如何建立一個簡單的 RESTful API。

首先到 Lombok 官網 Download (<https://projectlombok.org/download.html>) 頁面下載 Eclipse 所需要的套件 `lombok.jar`



首先，確保你的專案中包含了相關的相依套件，例如 Spring Boot 和 Spring Web。

```
<!-- Spring Boot 相依套件 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

```
<!-- Spring Web 相依套件 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

```
<!-- Lombok 相依套件 -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.28</version>
    <scope>provided</scope>
</dependency>
```

接下來，我們將建立一個代表使用者的實體類 User，並使用 Lombok 的注釋來減少冗長的程式碼。

```
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
```

```
@Data
@NoArgsConstructor
@AllArgsConstructor
```

```
public class User {
    private String id;
    private String name;
    private int age;
}
```

在這個範例中，我們使用了以下 Lombok 注釋：

@Data：這個注釋自動為類生成了 `getter`、`setter`、`equals`、`hashCode` 和 `toString` 方法。

@NoArgsConstructor：這個注釋生成了一個無參數的建構函式。

@AllArgsConstructor：這個注釋生成了一個包含所有參數的全參數建構函式。

接下來，我們將建立一個 REST 控制器來處理 HTTP 請求。

```
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    private List<User> userList = new ArrayList<>();

    @GetMapping
    public List<User> getAllUsers() {
        return userList;
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable String id) {
        for (User user : userList) {
            if (user.getId().equals(id)) {
                return user;
            }
        }
        return null;
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        userList.add(user);
        return user;
    }
}
```

```

    @PutMapping("/{id}")
    public User updateUser(@PathVariable String id, @RequestBody User
updatedUser) {
        for (int i = 0; i < userList.size(); i++) {
            User user = userList.get(i);
            if (user.getId().equals(id)) {
                userList.set(i, updatedUser);
                return updatedUser;
            }
        }
        return null;
    }
}

```

```

    @DeleteMapping("/{id}")
    public User deleteUser(@PathVariable String id) {
        for (User user : userList) {
            if (user.getId().equals(id)) {
                userList.remove(user);
                return user;
            }
        }
        return null;
    }
}

```

在這個範例中，我們使用了不同的 HTTP 方法處理不同的請求：

@GetMapping：處理 GET 請求，用於獲取所有使用者或特定使用者。

@PostMapping：處理 POST 請求，用於創建新使用者。

@PutMapping：處理 PUT 請求，用於更新現有使用者。

@DeleteMapping：處理 DELETE 請求，用於刪除使用者。

在這個範例中，我們使用了 **User** 類作為請求主體的資料類型，以及 **PathVariable** 和 **RequestBody** 注釋來接收路徑參數和請求主體的資料。

現在，你可以運行 **Spring Boot** 應用程式，並通過發送 HTTP 請求來測試這個 RESTful API。根據路由設置，你可以使用不同的端點來執行相應的操作。